

Anàlisi i Recomanacions per a un Algorisme d'Empaquetament 3D Irregular en Python

Aquest informe presenta una anàlisi exhaustiva i recomanacions estratègiques per al desenvolupament d'un sistema robust d'empaquetament 3D per a objectes irregulars. Basant-se en les especificacions proporcionades, que inclouen models STL complexos, rotació lliure, múltiples modes d'empaquetament i l'ús de biblioteques com **trimesh**, **pyvista** i **pybullet**, aquest document s'enfoca a abordar els reptes tècnics més importants: la detecció de col·lisions precises, l'optimització de l'empaquetament i la garantia de l'estabilitat física. L'anàlisi es basa exclusivament en les fonts d'informació proporcionades, assegurant la fidelitat i la integritat dels coneixements presents.

Disseny de Mecanismes de Detecció de Col·lisions Precises i Eficients

La detecció de col·lisions és el nucli operatiu de qualsevol sistema d'empaquetament 3D, especialment quan es treballa amb objectes irregulars i orientacions lliures. El nivell de precisió necessari depèn directament del mode d'operació del sistema, i una implementació eficient i robusta és essencial per equilibrar la qualitat de l'empaquetament amb el temps de càlcul disponible. Les fonts analitzades indiquen tres mecanismes clau: Oriented Bounding Boxes (OBB) per a una detecció ràpida i precisa, el Teorema de l'Eix Separador (SAT) per a la seva implementació, i l'ús de la geometria real de la malla per a simulacions físiques.

L'Oriented Bounding Box (OBB) emergeix com la representació ideal per a objectes arbitràriament rotats, superant les limitacions de les caixes alineades als eixos (AABB) que no canvien de forma amb la rotació⁴. Una OBB pot ajustar-se millor a la geometria d'un objecte, oferint una detecció de col·lisions més precisa que una AABB sense arribar a la complexitat de la comparació puntual de vèrtexs⁶. La implementació d'aquest mecanisme requereix una estructura de dades que inclogui la posició central (**Pos**), els tres vectors d'eixos d'orientació (**AxisX**, **AxisY**, **AxisZ**) i la meitat de la mida de cada costat (**Half_size**)². Aquest enfocament permet modelar la caixa envoltant d'un objecte independentment de la seva orientació espacial, un requisit fonamental per a l'optimització de la densitat en múltiples direccions⁸.

Per a la detecció de col·lisions entre dues OBB en 3D, el mètode estàndard de la indústria és el Teorema de l'Eix Separador (Separating Axis Theorem - SAT). Aquest teorema estableix que si es pot trobar un pla (o eix en 3D) que separi completament dos objectes convexos, llavors no hi ha cap col·lisió^{2 6}. Per a OBB, cal provar una sèrie específica d'eixos potencialment separadors. L'anàlisi indica que cal verificar fins a 15 eixos: els tres eixos d'orientació de la primera OBB, els tres de la segona, i nou eixos resultants del producte vectorial entre cada parell d'eixos (un de cada OBB)^{2 6 8}. L'omissió dels nou productes creuats pot portar a falsos positius, ja que es podrien perdre interseccions tangencials o molt properes entre cares de diferents objectes⁸. La implementació

computacional d'aquest test es basa en projeccions escalars. Per a cada un dels 15 eixos, es projecten sobre ell les dues OBB i es calculen els seus intervals de projecció (mínim i màxim). Si hi ha alguna superposició entre aquests intervals, l'eix no és un eix separador. Només si tots els 15 eixos mostren una superposició es pot concloure que no hi ha cap pla de separació i, per tant, hi ha una col·lisió². Es pot estimar un cost aproximadament de 200 operacions de coma flotant (FLOPS) per a aquesta verificació completa⁶. Per optimitzar, es pot recórrer a jerarquies de volums envolvents, on es realitza una prova SAT a gran escala amb OBB simples abans de passar a tests més detallats i costosos⁶.

Un aspecte crític de l'implementació SAT robusta, especialment en simulacions dinàmiques, és gestionar la precisió numèrica i evitar "feature flip-flops" (canvis ràpids de característiques de contacte). Dirk Gregorius, un expert en simulació de física, recomana l'ús de toleràncies relatives i absolutes (**kRelFaceTol**, **kAbsTol**, **kRelEdgeTol**) per donar prioritat a les característiques de contacte més significatives, com les cares, sobre les arestes o els vèrtexs¹. Aquesta pràctica evita oscil·lacions numèriques i garanteix una resposta més estable del motor de física. Per a modes d'empaquetament que utilitzen simulació física, com el que fa servir PyBullet, la precisió de la detecció de col·lisions esdevé primordial. En aquest cas, el sistema ha de recórrer a la geometria real de les malles STL per a la detecció. Això implica que, encara que es facin servir OBB per a la cerca accelerada i la selecció inicial d'objectes propers, el test final de col·lisió hauria de ser realitzat per la biblioteca de física subyacent (PyBullet), que probablement utilitza algorismes més sofisticats com GJK (Gilbert-Johnson-Keerthi)⁵. És crucial comprendre que aquesta integració requereix que la geometria de la malla estigui correctament configurada i accessible des de l'entorn de simulació de PyBullet.

Estratègies Avançades d'Optimització de l'Empaquetament

Una vegada s'ha implementat un mecanisme de detecció de col·lisions robust, el següent pas consisteix a desenvolupar algorismes d'empaquetament que maximitzin la densitat dins del contenidor. El problema general d'empaquetar objectes irregulars, conegut com a "bin packing", és NP-hard, el que significa que no existeixen solucions tractables per a grans instàncies en temps polinòmic. Per tant, la cerca d'una bona solució requereix l'ús d'algorismes heurístics i metaheurístics. Basant-nos en les funcionalitats del sistema actual, podem analitzar i proposar estratègies per als diversos modes d'empaquetament.

El mode "Floor Mode" es descriu com un empaquetament organitzat en pisos amb separació configurable. Aquest tipus d'estructura suggerix un enfocament "guillotine-like" o de "fila i columna". Una possible estratègia seria iterar sobre la pila d'objectes a empaquetar, ordenats per una certa característica (per exemple, àrea de base decreixent). Per a cada objecte, es buscaria la fila i la columna adequades dins d'un pis actual per tal d'introduir-lo sense col·lidir amb els objectes ja situats. La construcció d'un nou pis es produiria quan no hi hagués cap ubicació viable a la part inferior. Aquest mètode, encara que simple, pot funcionar bé per a certes classes d'objectes i generar patrons visuals organitzats. No obstant això, la seva eficiència depèn fortament de l'ordenació inicial dels objectes i pot deixar molts buits a les cantonades.

El mode "Bulk Mode" permet un empaquetament lliure amb detecció de col·lisions. Aquesta llibertat permet l'ús d'algorismes més complexos i adaptatius. Un mètode força potent i popular per a

aquest tipus de problemes és l'algorisme de localització d'objectes aleatòria (RVO), que s'ha utilitzat amb èxit en aplicacions similars. Aquest algorisme funciona inserint objectes de manera aleatòria en el contenidor i permetent que cada objecte "s'estiri" o es mogui lliurement fins a tocar-ne un altre o el límit del contenidor. Aquest procés de relaxació física, encara que no sigui realista, ajuda a compactar els objectes i a reduir els buits. Alternativament, es podria utilitzar un mètode de tipus "first fit" o "best fit" adaptatiu, on cada cop s'insereix un objecte, es busca la millor ubicació disponible basant-se en criteris com la proximitat a altres objectes o la minimització de l'espai residual. L'ús de l'OBB per calcular les dimensions òptimes de cada peça en funció de la seva orientació lliure és un avantatge significatiu en aquest mode, ja que permet explorar múltiples configuracions per trobar la que ofereix el major acostament.

Finalment, el mode "OBB Mode", que optimitza basant-se en Oriented Bounding Boxes, sembla implicar una cerca més exhaustiva. Aquest mode probablement implícit un processament pre-processament on, abans de l'empaquetament principal, es calcula un conjunt discret d'orientacions alternatives per a cada objecte. Per a cada objecte, es podria rotar sistemàticament al voltant d'un eix o utilitzar una cerca per veïns més cercans per trobar una orientació que minimitzi la mida de la seva OBB circumscrita. Un cop aquest conjunt d'orientacions "optimistes" s'ha generat, es pot procedir amb qualsevol dels algorismes d'empaquetament anteriorment mencionats, però seleccionant només entre aquestes pre-computades orientacions. Aquest enfocament transforma parcialment el problema 3D d'orientació en un problema combinatòri de tria d'orientació i posició, que pot ser resolt de manera més eficient. Aquesta estratègia combina la flexibilitat de la rotació lliure amb l'eficiència d'un espai de cerca discretitzat.

Mode d'Empaquetament	Descripció Clau	Estratègia Recomanada	Justificació
Floor Mode	Empaquetament organitzat en pisos.	Algorisme guillotine-like o de fileres/columnes.	Genera patrons organitzats i controlats, adequat per a sistemes que busquen una distribució estructurada.
Bulk Mode	Empaquetament lliure amb detecció de col·lisions.	Algorisme de localització d'objectes aleatòria (RVO) o cerca adaptativa ("first/best fit").	Maximitza la densitat explorant de manera eficient l'espai lliure i permetent moviments locals per compactar.
OBB Mode	Optimització basada en OBB.	Pre-computació d'orientacions òptimes i posterior empaquetament.	Comprimeix l'ample espai d'orientacions a un conjunt finit de candidates, facilitant la cerca d'una solució òptima.

Integració de Simulació Física per a Estabilitat i Realisme

Garantir l'estabilitat física dels objectes empaquetats és un objectiu secundari crític, especialment en modes que simulem el món real. L'estabilitat no només millora la credibilitat visual, sinó que també pot tenir implicacions pràctiques si el sistema s'utilitza per a la planificació de càrrega o la manipulació robòtica. L'estratègia principal per aconseguir-ho, tal com suggereix el sistema actual, és l'ús d'un motor de simulació física com PyBullet .

L'aproximació bàsica consisteix a omplir el contenidor amb tots els objectes a empaquetar, posicionats inicialment segons una solució d'empaquetament generada pel seu algorisme preferit (potser un mode "Bulk Mode"). A continuació, es llegeix l'estat d'aquests cossos rígids i es comença la simulació. Durant la simulació, els objectes interactuen segons les lleis de la física: reben gravetat, experimenten fricció en contacte amb el fons i entre ells, i poden rodolar o desplaçar-se fins a trobar una configuració d'equilibri estable. Una vegada transcorregut un període de temps de simulació suficient o quan el sistema entra en repòs (quan la velocitat i l'acceleració de tots els objectes romanen per sota d'un cert umbral durant un interval de temps), es para la simulació i es guarda l'estat final. Aquest estat resultant representa una configuració d'empaquetament molt més estable i realista.

No obstant això, aquesta aproximació té els seus reptes. El primer i més important és el temps de càlcul. Una simulació física robusta pot ser molt costosa computacionalment, especialment si es tracta de milers d'objectes complexos. Cal mantenir un bon equilibri entre la qualitat de l'estabilitat i el temps disponible per a la planificació. Una estratègia pot ser limitar el temps de simulació o fer-la en passes curtes per obtenir una estabilitat parcial.

El segon repte resideix en la detecció de col·lisions. Com es va discutir anteriorment, per a una simulació realista, la detecció de col·lisions ha de ser precisa. Per tant, mentre la cerca accelerada i l'assignació inicial d'objectes es poden fer amb OBB per rapidesa, el motor de física (PyBullet) necessitarà la geometria real de les malles per a la detecció de contactes durant la simulació . Això vol dir que cal un flux de dades coherent entre el codi d'empaquetament i l'entorn de simulació de PyBullet. A més, cal gestionar la fricció i els coeficients de restitució adequadament per tal que el comportament de la simulació reflecteixi la realitat el més fidelment possible.

Finalment, l'estabilitat no es mesura només en termes de repòs. Un objecte pot estar en repòs mecànicament (forces netes zero), però pot ser dinàmicament inestable (per exemple, un cub sobre un vèrtex). Per detectar aquest tipus d'inestabilitat, caldria realitzar una anàlisi post-simulació per determinar si qualsevol objecte té un únic punt de contacte o si la seva base de suport és excessivament petita. Si es detecta una inestabilitat d'aquest tipus, es podria intentar introduir una nova peça per sostenir-lo o marcar l'empaquetament com a no-vàlid i tornar a intentar una altra configuració. Dirk Gregorius també insisteix en la importància de la qualitat del contacte en simulacions; una bona qualitat de contacte, gestionada mitjançant toleràncies, contribueix a una estabilitat global millor de tot el sistema ¹ .

Implementació Pràctica amb Biblioteques Python: Trimesh, Pyvista i Pybullet

L'elecció de les eines de programari és un factor determinant en la viabilitat i l'eficiència del projecte. El sistema actual fa servir una combinació de biblioteques de Python com **trimesh**, **pyvista** i **pybullet**. Cada una d'aquestes biblioteques té un paper clar i complementari en el flux de treball d'empaquetament i simulació.

trimesh és una eina extremadament poderosa i versàtil per a la gestió de geometria 3D a partir de malles triangulars. Sembla ser la base per a la major part del processament de l'objecte. Proporciona funcionalitats crucials com:

- * Càrrega i simplificació de malles: Pot llegir formats de fitxer estàndard com STL i ofereix eines avançades per simplificar la topologia d'una malla conservant la seva forma general, tal com es menciona en la pregunta inicial. Això és essencial per a l'optimització de rendiment.
- * Anàlisi de geometria: Calcula propietats geomètriques importants com el centre de massa, l'el·lipsoide d'inèrcia i la caixa envoltant orientada (OBB) mínima. Aquest últim és particularment rellevant, ja que pot substituir la necessitat de codificar l'algorisme de càlcul de l'OBB des de zero, estalviant temps i reduint el risc d'errors ⁶.
- * Intersecció de malles: Ofereix funcions per a detectar interseccions entre malles, que es poden utilitzar per a proves de detecció de col·lisions precises. Tot i que pot no ser tan ràpid com un algorisme SAT específic per a OBB, pot ser útil per a validacions o per a casos d'interès complexos que escapen a la convexitat de les OBB.

* Transformacions 3D: Gestiona rotacions, translacions i escalats de manera eficient, essencial per a canviar l'estat d'un objecte durant l'algorisme d'empaquetament.

pyvista és una biblioteca per a la visualització científica en 3D que pot ser invaluable durant el desenvolupament i la depuració. Encara que no està explícitament citat en les fonts proporcionades, el seu ús en un projecte que fa servir **trimesh** i **pybullet** és natural. Pot ser utilitzat per:

- * Visualitzar l'estat inicial i final: Mostrar el contenidor, els objectes a empaquetar i la solució trobada per ajudar a validar visualment l'algorisme.
- * Animar la simulació: Fer una animació de la fase de relaxació o la simulació física per entendre el comportament dels objectes i identificar possibles problemes d'estabilitat.
- * Debugging: Resaltar regions d'interès, com ara zones de contacte o interseccions detectades, per a una anàlisi més profunda.

pybullet és el motor de física de codi obert que es farà servir per a la simulació de l'estabilitat. El seu paper és clar: gestionar la dinàmica del sistema després que s'hagi col·locat una disposició d'objectes. El flux típic d'ús seria:

1. Crear un entorn de Bullet.
2. Per a cada objecte a empaquetar, creat-ne un cos rígid a l'entorn de Bullet a partir de la seva geometria (probablement una malla).
3. Posiciona tots aquests cossos a l'estat inicial determinat pel seu algorisme d'empaquetament.
4. Activa la simulació i permet que es relaxin fins a trobar un estat d'equilibri.
5. Rebràs l'estat final dels objectes i podràs recuperar la seva posició i orientació per a la sortida.

L'integració entre **trimesh** i **pybullet** és crucial. **trimesh** pot preparar i proporcionar la geometria i les propietats físiques (massa, inèrcies) mentre **pybullet** gestiona la part dinàmica. Aquesta combinació permet un desenvolupament modular i eficient, on cada biblioteca es dedica a la seva tasca específica amb excel·lència. L'exemple en JavaScript de detecció de col·lisions entre formes límits i cossos físics es pot considerar una analogia conceptual de com **trimesh** (formes límits) i **pybullet** (cossos físics) interactuaran en Python ⁴.

Gestió de Complexitat Geomètrica i Rendiment Óptim

Treballar amb models 3D STL complexos planteja reptes significatius en termes de rendiment i gestió de memòria. La complexitat de la geometria afecta directament la velocitat de les operacions de detecció de col·lisions i simulació física. Per aconseguir un sistema eficient, cal adoptar estratègies actives de gestió de complexitat.

La font principal ja menciona que es té un programa per a reduir la complexitat dels STL, cosa que demostra una consciència preexistent del problema. Aquest procés, conegut com a simplificació de malles, és un punt de partida essencial. Biblioteques com **trimesh** ofereixen algorismes de simplificació que redueixen el nombre de cares (triangles) d'una malla mantenint una aparença visual similar. Aquesta etapa previ-pròcs ha de ser part integrant del flux de treball: quan un nou model s'afegeix al sistema, ha de passar automàticament per un filtre de simplificació abans de ser emmagatzemat o utilitzat per a l'empaquetament. La decisió sobre el nivell de simplificació (la proporció de cares a eliminar) serà un paràmetre clau que caldrà ajustar per trobar l'equilibri entre precisió i rendiment.

Tanmateix, la simplificació de malles no sempre és suficient per a una detecció de col·lisions ràpida. Aquí, l'ús d'una jerarquia de volums envolvents es converteix en una estratègia de major ordre ⁶. En comptes d'executar una comparació cara-a-cara (vertex-to-vertex) entre dues malles complexos, es pot construir una jerarquia de volums envolvents més senzills (com OBB o esferes) que aproximem la geometria a diferents nivells de detall. Aquesta jerarquia permet un "pruning" ràpid de les bales: es pot començar la detecció de col·lisions comparant les caixes envolvents més grosses. Si aquestes no col·lideixen, es pot descartar immediatament que les malles subjacents ho fan, evitant càlculs innecessaris. Si hi ha intersecció, es pot descendre a nivells de detall més baixos de la jerarquia fins a arribar a la comparació de les malles originals si es confirma una possible col·lisió. Aquesta tècnica és fonamental per escalar el sistema a grans quantitats d'objectes.

Un altre aspecte de la gestió del rendiment és la caching o memorització. Molts càlculs en un algorisme d'empaquetament són repetitius. Per exemple, el cost computacional de calcular la OBB mínima per a un objecte en una orientació donada pot ser significant. Si aquest mateix objecte s'ha de provar en la mateixa orientació múltiples cops durant l'execució de l'algorisme, seria molt ineficient recalcular-lo cada vegada. En lloc d'això, es pot crear una taula hash (diccionari en Python) que associi cada combinació d'objecte i orientació a la seva OBB corresponent. Abans de calcular, l'algorisme miraria primer a la taula de cache; si el resultat ja hi és, el reutilitzaria. Aquesta tècnica, coneguda com a memoization, pot reduir dràsticament el temps total de càlcul, especialment en algorismes que exploren exhaustivament un gran espai de solucions.

Finalment, l'optimització del codi en si mateix és crucial. Encara que es facin servir les estructures de dades i algorismes correctes, una implementació deficiente pot ser lent. Es recomana prioritzar l'ús de funcions vectoritzades i operacions massives allà on sigui possible, especialment amb l'ajuda de biblioteques com NumPy. En lloc de fer bucles **for** en Python per iterar sobre vèrtexs o cares, es poden processar blocs de dades de cop. A més, es pot considerar l'ús d'enginyeria de compilació com Cython o Numba per a les parts més crítiques del codi (com les funcions de detecció de col·lisions bàsiques) per a traduir-les a C i obtenir un augment significatiu de la velocitat.

Recomanacions Estratègiques i Resum de l'Implementació

En conclusió, el desenvolupament d'un sistema robust d'empaquetament 3D per a objectes irregulars, com el que s'especifica, és un projecte complex però completament factible amb l'enfocament i les eines adequades. L'anàlisi de les fonts disponibles i el context proporcionat punten cap a un disseny modular i jeràrquic que equilibra precisió, rendiment i estabilitat. Les següents recomanacions estratègiques resumiran les decisions clau per a la implementació:

1. **Arquitectura Modular:** Mantenir una separació clara de responsabilitats entre les capes del sistema. Utilitzar **trimesh** per a la gestió i anàlisi de la geometria (lectura, simplificació, càlcul d'OBB), un motor d'empaquetament central per a la lògica d'assignació d'objectes als contenidors, i **pybullet** per a la simulació física i la validació d'estabilitat. Aquesta modularitat permet desenvolupar i depurar cada component de forma independent i facilita futures millories o canvis.
2. **Jerarquia de Col·lisions:** Implementar una jerarquia de detecció de col·lisions. L'etapa inicial (pre-procesament o cerca accelerada) hauria de fer servir OBB i el Teorema de l'Eix Separador (SAT) per a una detecció ràpida i eficient ^{2 6}. Només quan es detecti una intersecció potencial entre dos objectes es passaria a una prova més costosa i precisa utilitzant la geometria de la malla real, delegant això en **trimesh** o directament en **pybullet** per a la fase de simulació.
3. **Algorismes d'Empaquetament Evolutius:** Començar amb algorismes més simples i eficients per als modes "Floor Mode" i "Bulk Mode". Per a una millora gradual, incorporar heurístiques més avançades com l'algorisme RVO o cerques adaptatives. Per al mode "OBB Mode", dedicar recursos addicionals a un pre-procesament que calcule un conjunt discret d'orientacions optimistes per a cada objecte, convertint el problema de rotació contínua en un problema combinatòri més tractable.
4. **Simulació Física per a l'Estabilitat:** Considerar la simulació física no només com una opció, sinó com un pas obligatori després de l'empaquetament inicial. Aquesta etapa, encara que computacionalment costosa, és la clau per a generar empacaments realistes i estables. Cal definir criteris clars per determinar quan una configuració està "estable" i gestionar adequadament la integració entre l'estat calculat i l'entorn de simulació de **pybullet**.
5. **Gestió Activa de la Complexitat:** Fer de la gestió de la complexitat geomètrica una prioritat. Automatitzar el procés de simplificació de malles i implementar una jerarquia de volums envolvents per a accelerar les comparacions ⁶. Aprofitar tècniques com el caching per a valors repetits i optimitzar el codi amb operacions vectoritzades per maximitzar el rendiment ⁷.

En resum, el camí cap a un sistema d'empaquetament 3D eficaç passa per una planificació detallada que aborde els reptes de la detecció de col·lisions, l'optimització de l'empaquetament i l'estabilitat física de manera coordinada. Centrant-se en una arquitectura modular que aprofite al màxim les fortalezes de les biblioteques existents (**trimesh**, **pybullet**) i adoptant estructures de dades i algorismes eficients (OBB, SAT, simulació física), es pot construir un sistema que no només maximitzi la densitat, sinó que també ho faci de manera robusta, eficient i realista.

en

1. 3D OBB vs OBB collision data? - Math and Physics - GameDev.net <https://www.gamedev.net/forums/topic/652667-3d-obb-vs-obb-collision-data/>
2. Simple Oriented Bounding Box OBB collision detection explaining <https://stackoverflow.com/questions/47866571/simple-oriented-bounding-box-obb-collision-detection-explaining>
3. 3D OBB vs OBB Collision Detection and Response <https://gamedev.stackexchange.com/questions/95251/3d-obb-vs-obb-collision-detection-and-response>
4. 3D collision detection - MDN - Mozilla https://developer.mozilla.org/en-US/docs/Games/Techniques/3D_collision_detection
5. Collision Detection in 2D or 3D – Some Steps for Success <https://randygaul.github.io/collision-detection/2019/06/19/Collision-Detection-in-2D-Some-Steps-for-Success.html>
6. [PDF] Collision Detection <https://www.cs.jhu.edu/~sleonard/cs436/collisiondetection.pdf>
7. <https://futur.upc.edu/RIS/tesis/t/SW50ZXJhY2Npw7M=> <https://futur.upc.edu/RIS/tesis/t/SW50ZXJhY2Npw7M=>
8. How many and which axes to use for 3D OBB collision with SAT <https://gamedev.stackexchange.com/questions/44500/how-many-and-which-axes-to-use-for-3d-obb-collision-with-sat>